

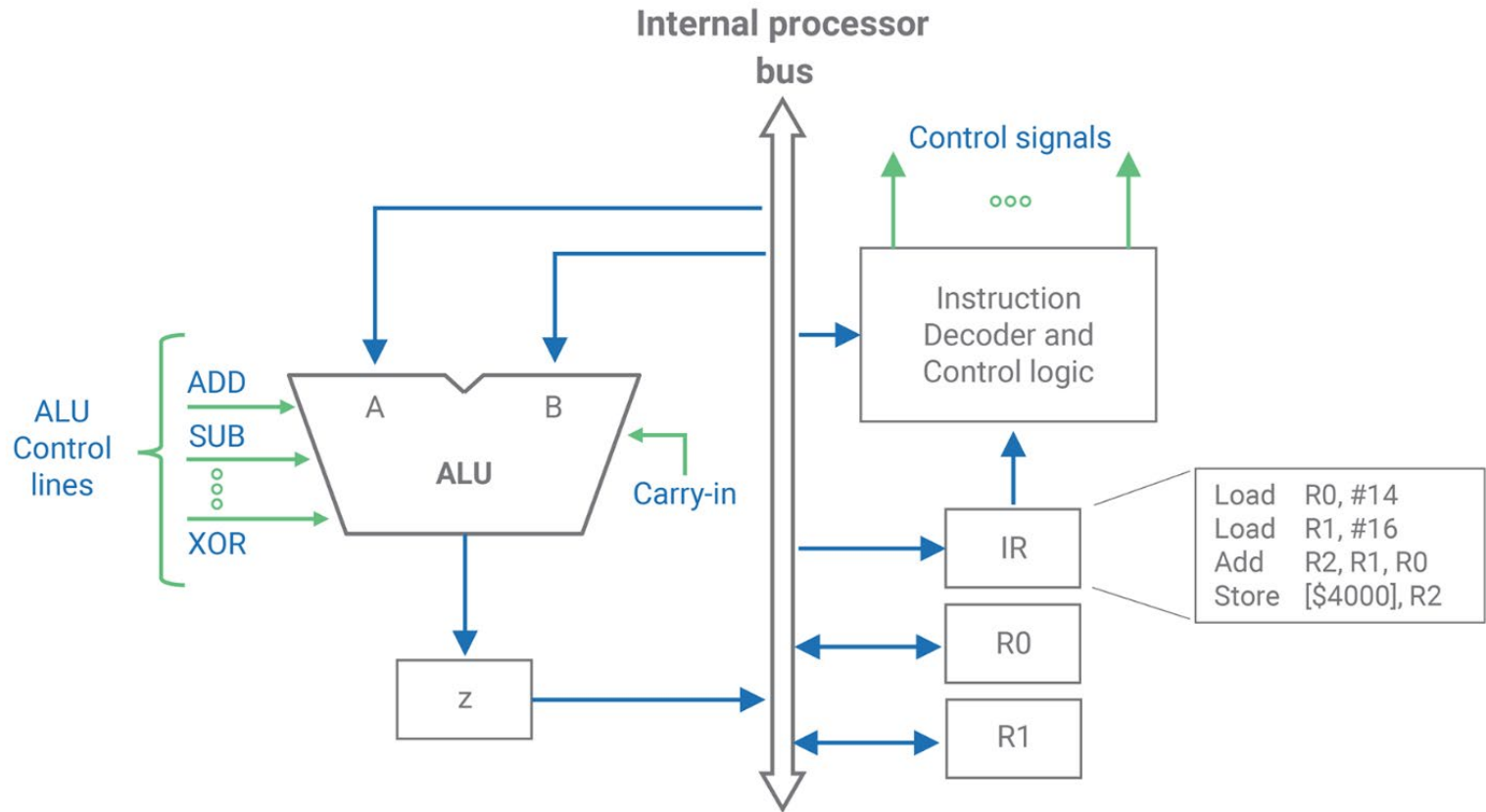
Lecture 6

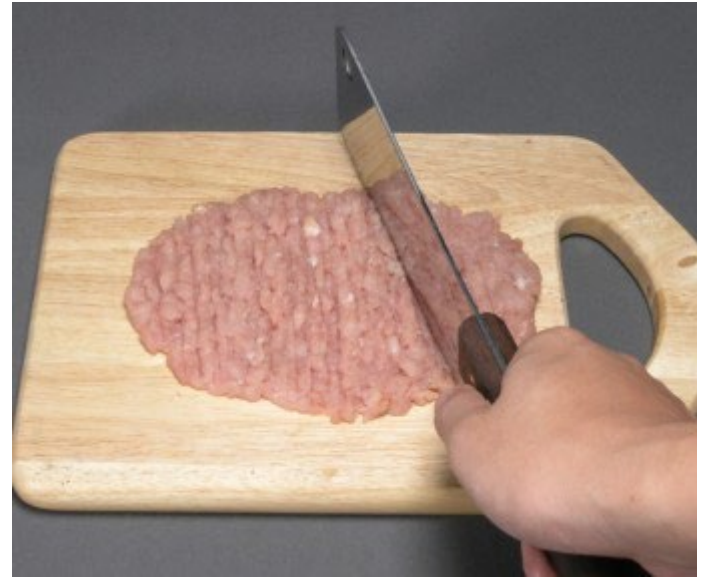
Computer Arithmetic

Previously ..

- Basic I/O
 - Memory Map I/O
 - Program Control I/O: Polling I/O vs Interrupt I/O
 - Interrupt Service Routine (ISR)
 - Interrupt for Multiple Devices: Int Vector, Int Nesting, Int Priority ..
- I/O Organization
 - I/O Interface: Address Decoder, Control Circuit, and Registers
 - Synchronous and Asynchronous Bus Protocols
 - Input and Output Interface Circuit

Central Processing Unit





ADD and SUB

Full Adder (1)

x_i	y_i	Carry-in c_i	Sum s_i	Carry-out c_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$s_i = x_i \oplus y_i \oplus c_i$$

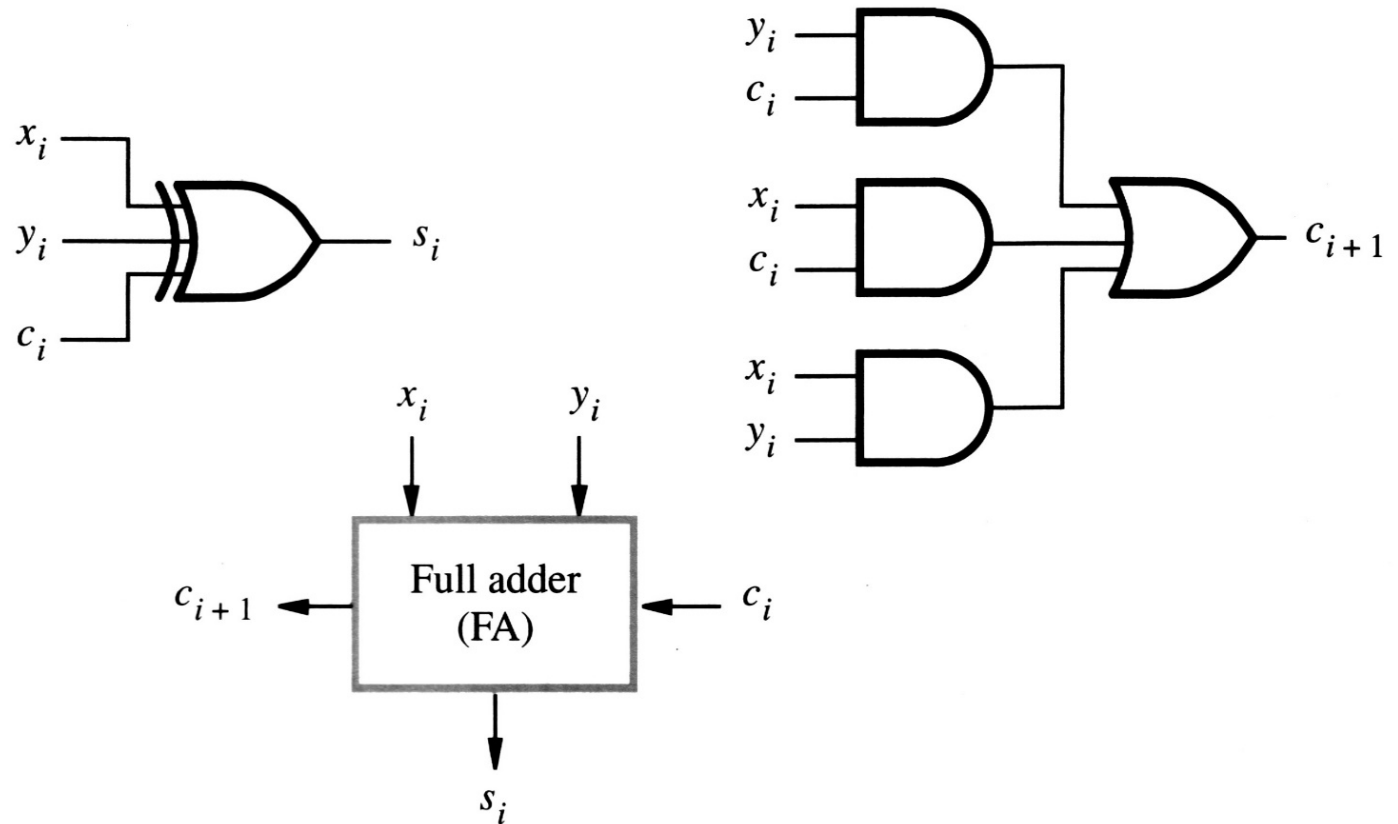
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$c_{i+1} = x_i c_i + y_i c_i + x_i y_i$$

Full Adder (2)

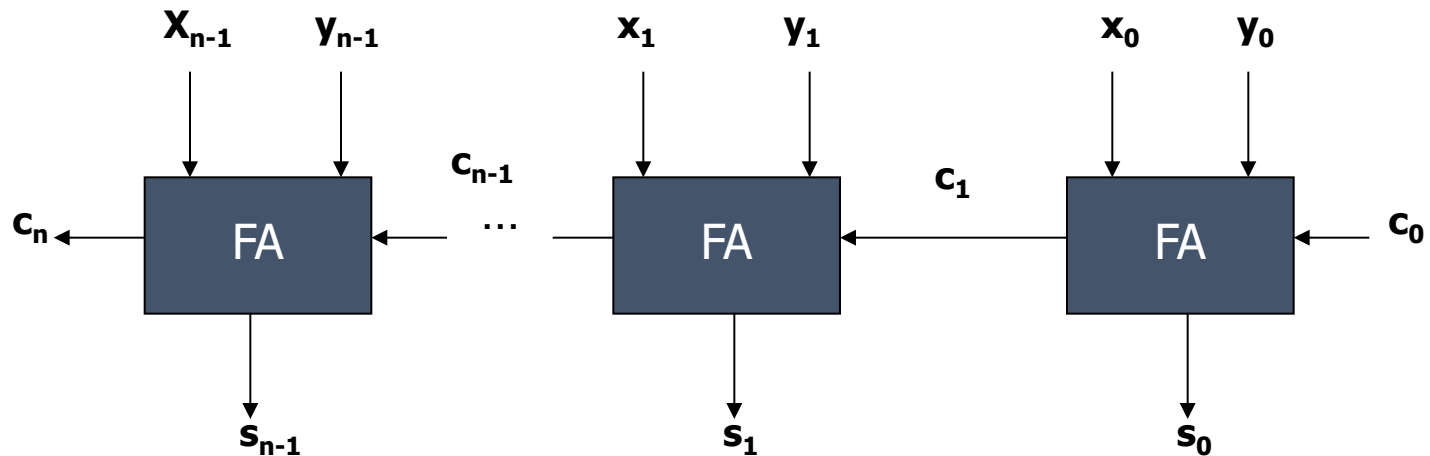
$$s_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = x_i c_i + y_i c_i + x_i y_i$$



Carry Ripple Adder (CRA)

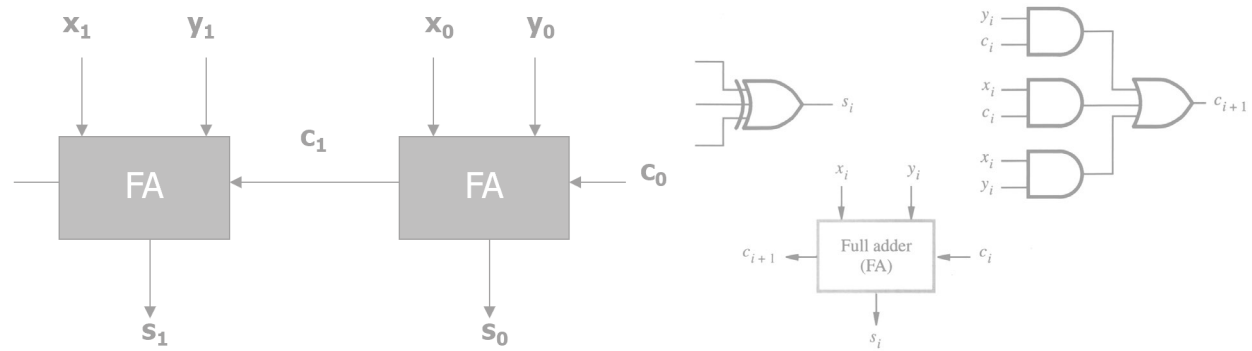
- An n -bit adder can be built by linking a series of full adders together and let the carry ripple through the circuit.



**Most significant bit
(MSB) position**

**Least significant bit
(LSB) position**

Fast Adder



- The carry-in of second adder is the carry-out of the first adder.
- Thus, the second adder can start adding only after the first adder is finished.
- The use of carry-ripple adder introduces a large amount of delay. Requires **2n gate delays** to produce the sum.
- To reduce the delay:
 - Use the fast electronic gate, which can be costly.
 - Use a different design, such as carry look-ahead adder (CLA).



The Carry Look Ahead Adder (1)

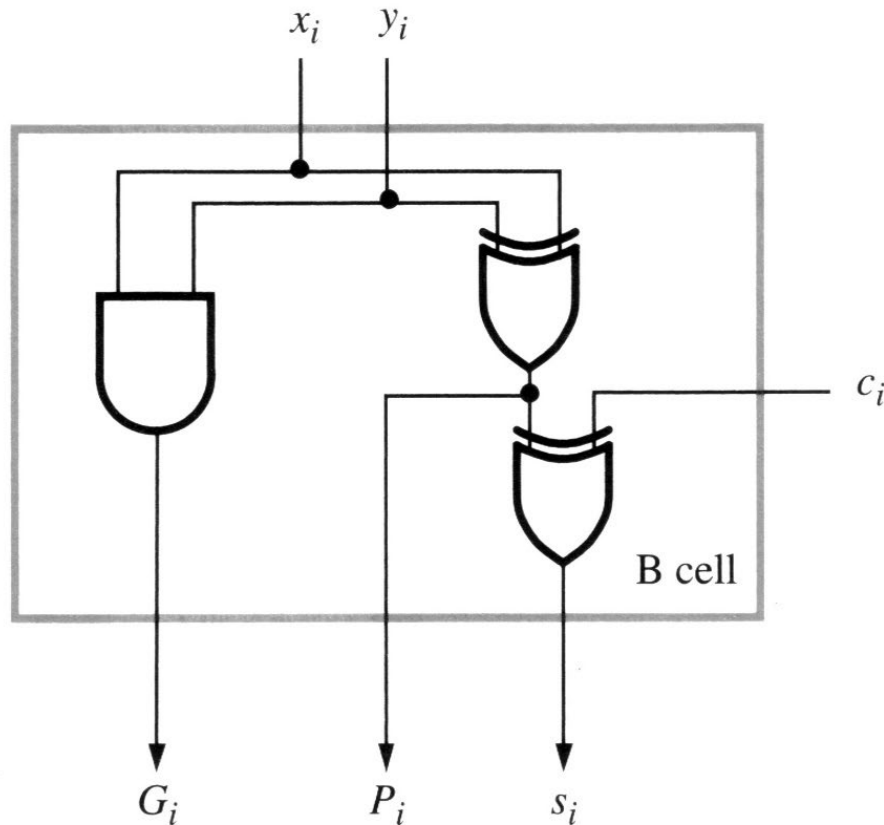
- $C_{i+1} = X_i C_i + Y_i C_i + X_i Y_i$
- $C_{i+1} = (X_i + Y_i)C_i + X_i Y_i$
- $X_i Y_i$ is called “generate function” or G_i for state i
- $X_i + Y_i$ is called “propagate function” or P_i for state i

- (1) If G_i is 1, then C_{i+1} is always 1 regardless of C_i
- (2) If P_i is 1, then $C_{i+1} = C_i$
- G_i and P_i can thus be used to predict the value of C_{i+1}

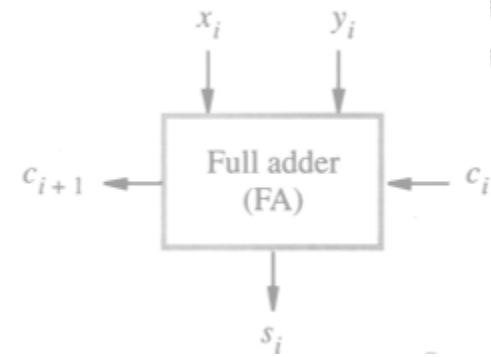
The Carry Look Ahead Adder (2)

$$s_i = x_i \oplus y_i \oplus c_i$$
$$c_{i+1} = x_i c_i + y_i c_i + x_i y_i$$
$$c_{i+1} = (x_i + y_i) c_i + x_i y_i$$

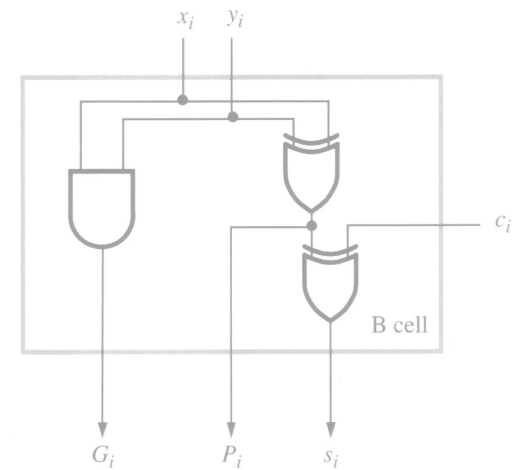
P_i G_i



(a) Bit-stage cell

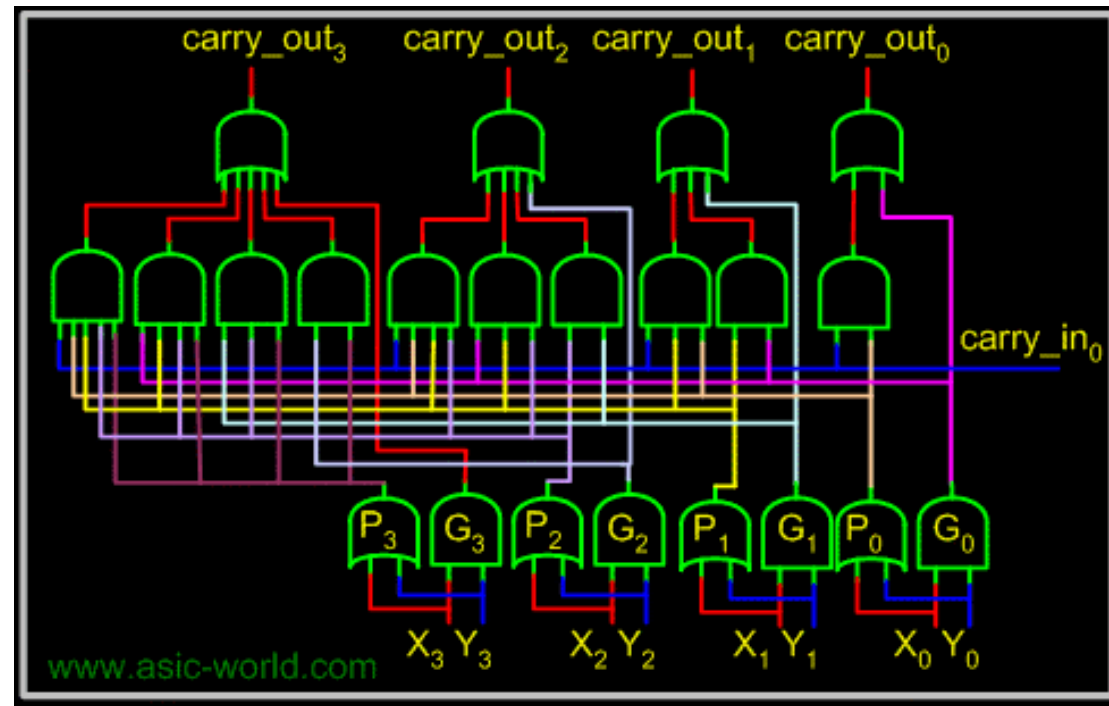


The Carry Look Ahead Adder (3)



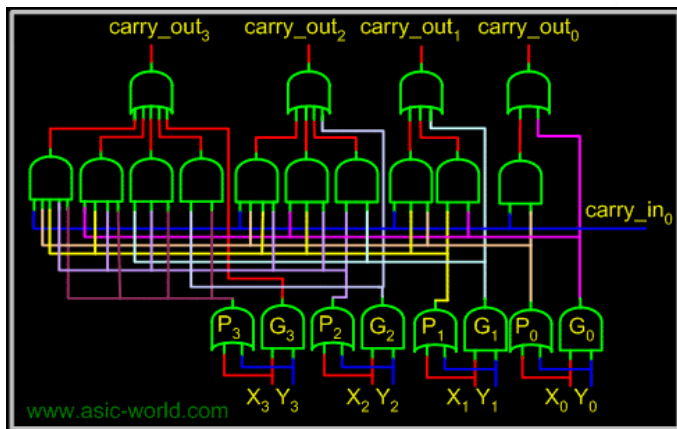
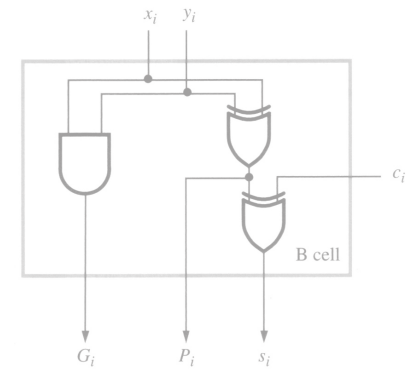
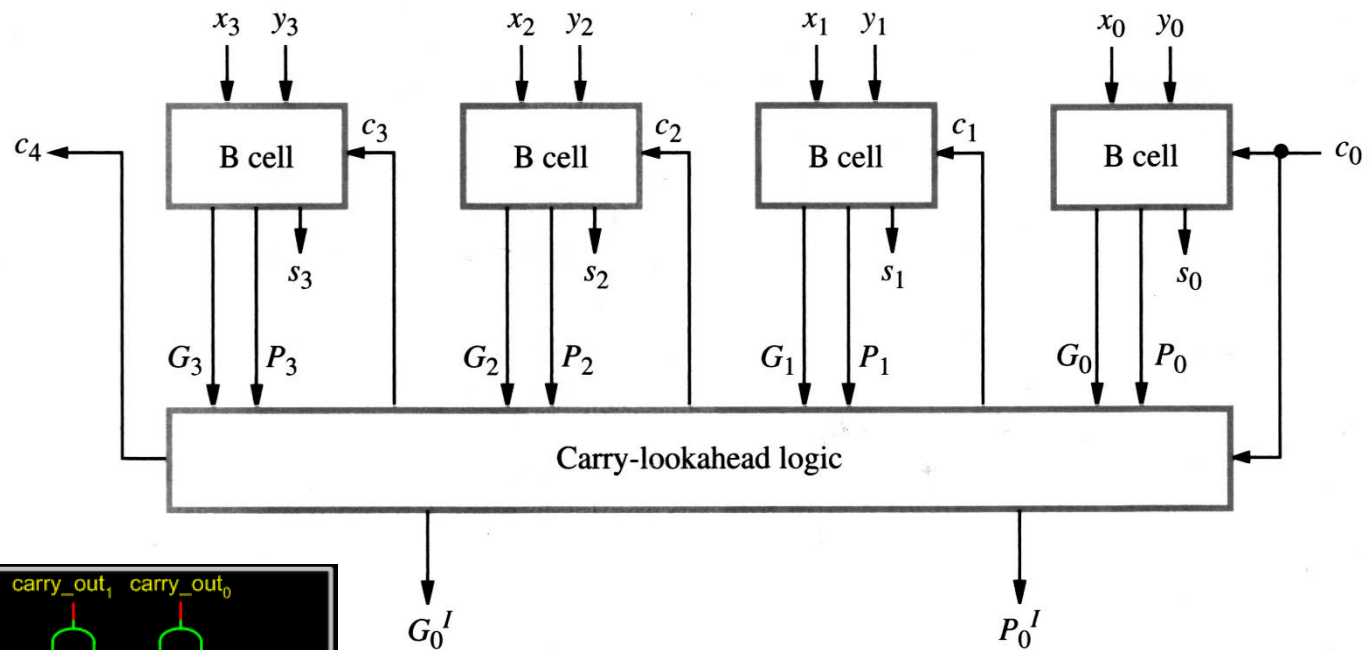
- S_i is derived from a 3-input xor-gate (which can be built from two xor-gates).
- Notice that: a xor gate is used instead of an or-gate in computing P_i . This can be done as a xor-gate only gives a different result from an or-gate, when $X_i = Y_i = 1$. In this case, G_i will be equal to 1 and the P_i value will not be needed.

The Carry Look Ahead Adder (4)



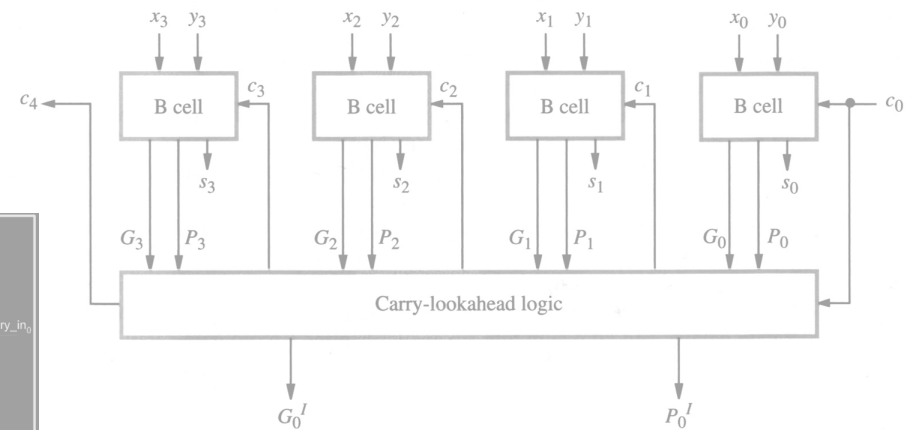
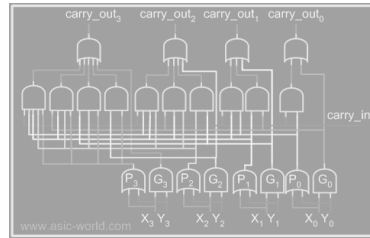
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1 C_1$
 $= G_1 + P_1 (G_0 + P_0 C_0)$
- $C_3 = G_2 + P_2 C_2$
 $= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
- $C_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \dots + P_i P_{i-1} \dots P_0 C_0$

The Use of CLA in 4-bit Adder

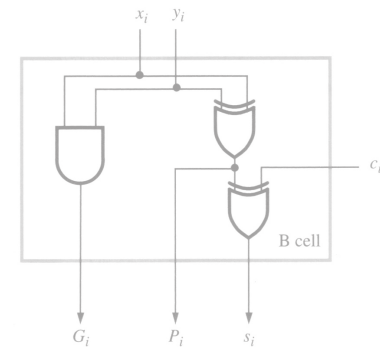


(b) 4-bit adder

The Carry Look Ahead Adder (5)



- Since the G and P values are available at the beginning of the computation, the carries of every bits can be computed in 3 gate delays after X, Y and C_0 enter the circuit.
 - One gate delay is needed to compute the P and G value
 - Two more gate delays are needed to compute the C_{i+1} value (AND-OR)
- S_i can be produced in one more gate delay after C_i is computed (S_i uses the total of 4 gate delays).

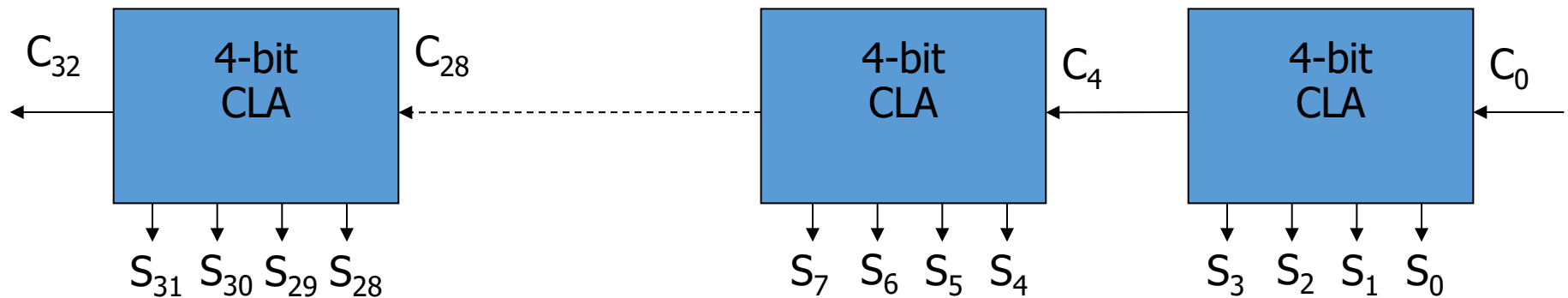


Comparison of Adders

- Carry-Look ahead Adder
 - Requires only 4 gate delays to produce the sum
 - Requires the use of n -input logic gates
 - Fast but expensive
- Carry-Ripple Adder
 - Requires $2n$ gate delays to produce the sum
 - Requires a set of 2-input logic gate
 - Slow but inexpensive

Example: Building An Adder (1)

- Build a 32-bit adder uses 8 of the 4-bit carry look ahead adders. Let the carry ripple through each 4-bit adder



Example: Building An Adder (2)

- C_4 is available after 3 gate delays
- C_8 is available after 5 gate delays
- C_{12} is available after 7 gate delays
- Thus, C_{32} will be available after $(7 \times 2) + 3 = 17$ gate delays
- All sum bits can be produced in $1 + 17 = 18$ gate delays.
- Using a 32 bit carry-ripple adder will require $32 \times 2 = 64$ gate delays

Example: 2-level CLA Circuit in 16-bit Adder

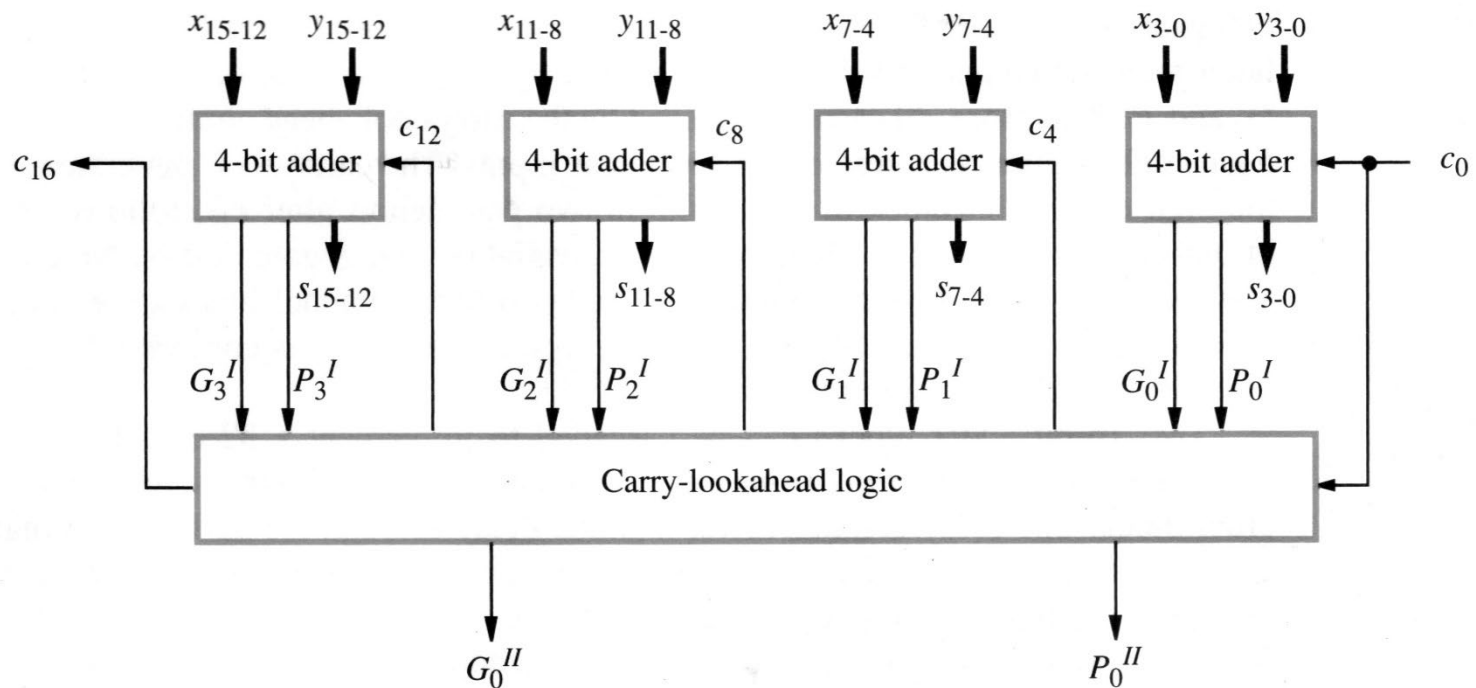
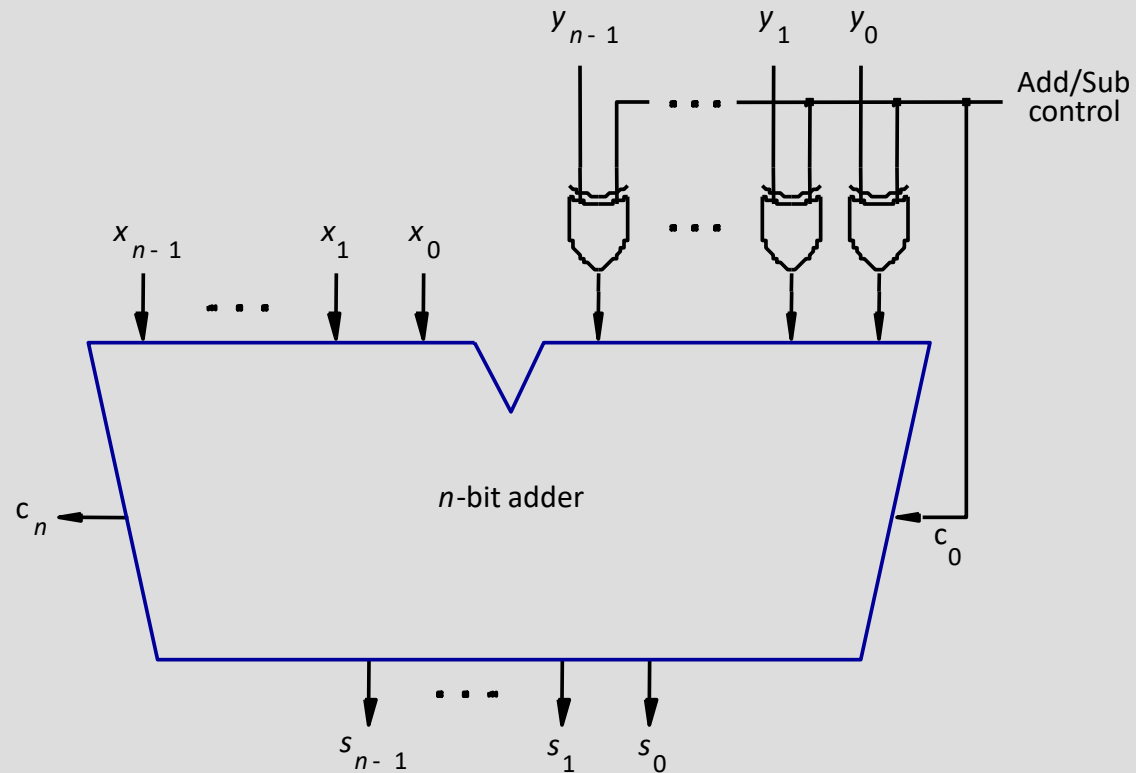


Figure 6.5 16-bit carry-lookahead adder built from 4-bit adders (see Figure 6.4b).

n -bit adder/subtractor



- Add/sub control = 0, addition.
- Add/sub control = 1, subtraction.



MUL and DIV

Tawan Thaeprasit to Everyone

มัลดีฟไม่ใช่เกาะหรือครับ

Unsigned Multiplication (1)

$$\begin{array}{r}
 1101 \\
 \times 1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 10001111
 \end{array}$$

(13) Multiplicand M

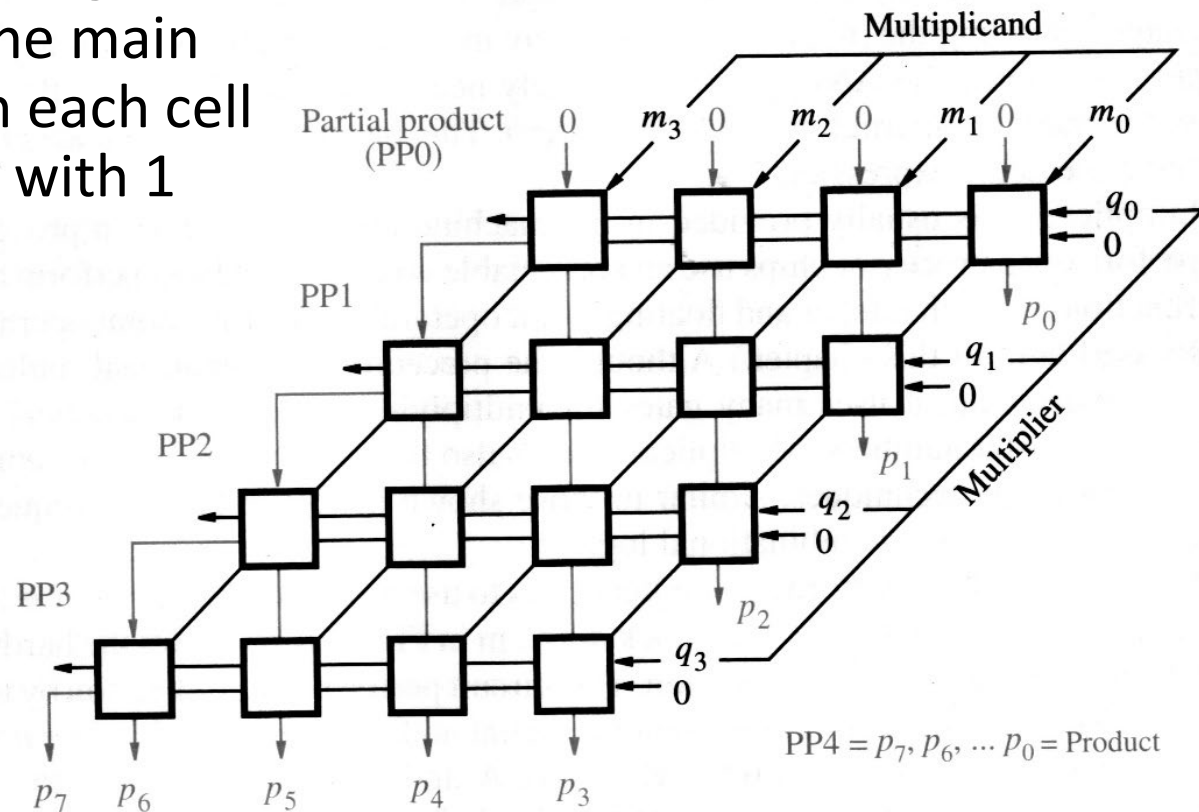
(11) Multiplier Q

(143) Product P

Unsigned Multiplication (2)

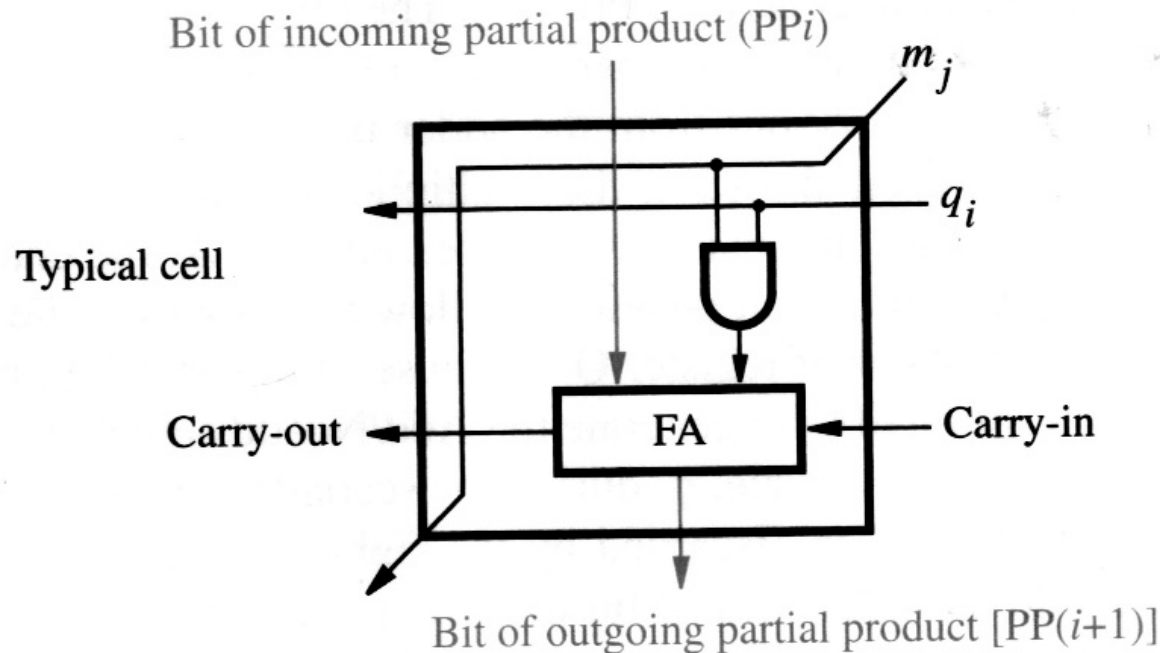
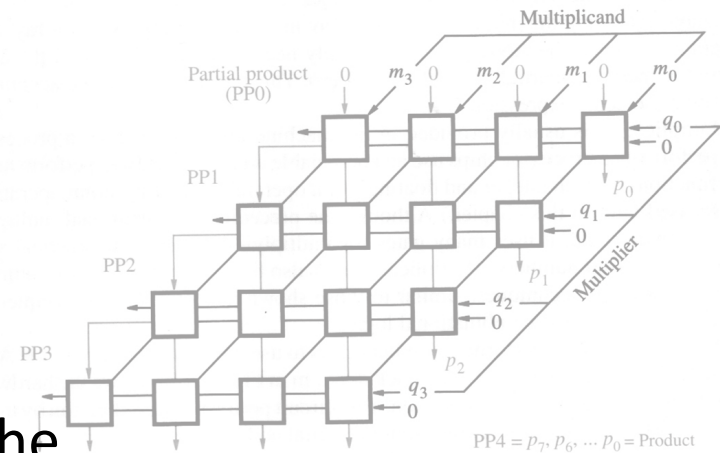
$$\begin{array}{r}
 \begin{array}{cccc}
 & 1 & 1 & 0 & 1 \\
 \times & 1 & 0 & 1 & 1 \\
 \hline
 & 1 & 1 & 0 & 1 \\
 & 0 & 0 & 0 & 0 \\
 & 1 & 1 & 0 & 1 \\
 \hline
 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1
 \end{array}
 \end{array}$$

Multiplication can be achieved by using a 2-D logic array. The main component in each cell is a full adder with 1 *and*-gate.

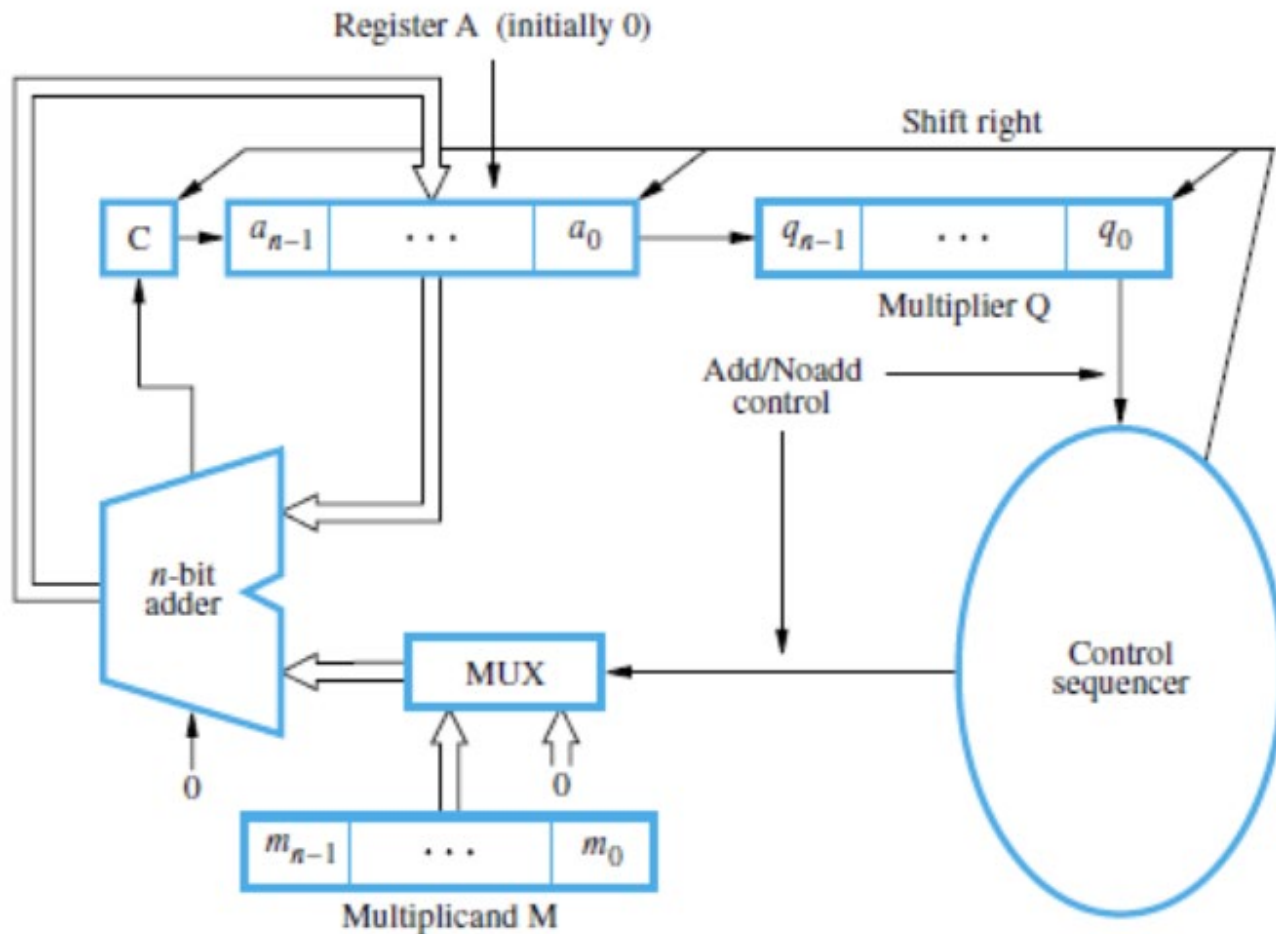


Unsigned Multiplication (3)

The *and*-gate determines whether a multiplicand bit, m_j , will be added to the incoming partial product bit, based on the value of the multiplier bit, q_i .

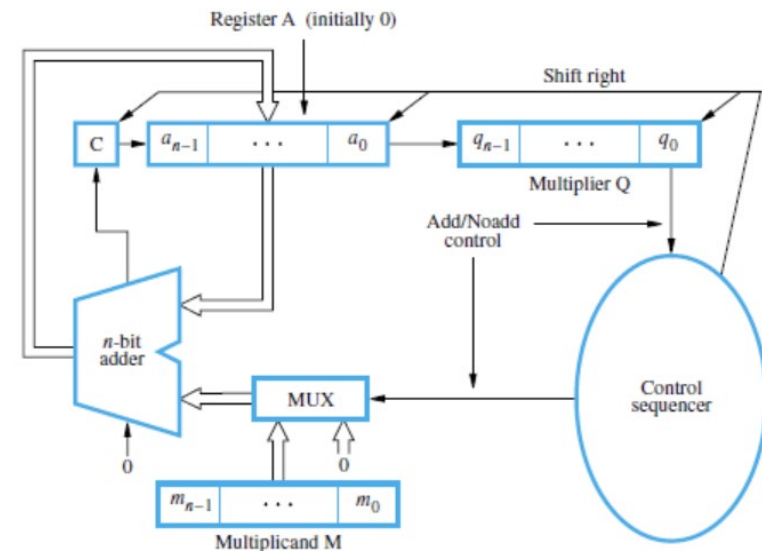
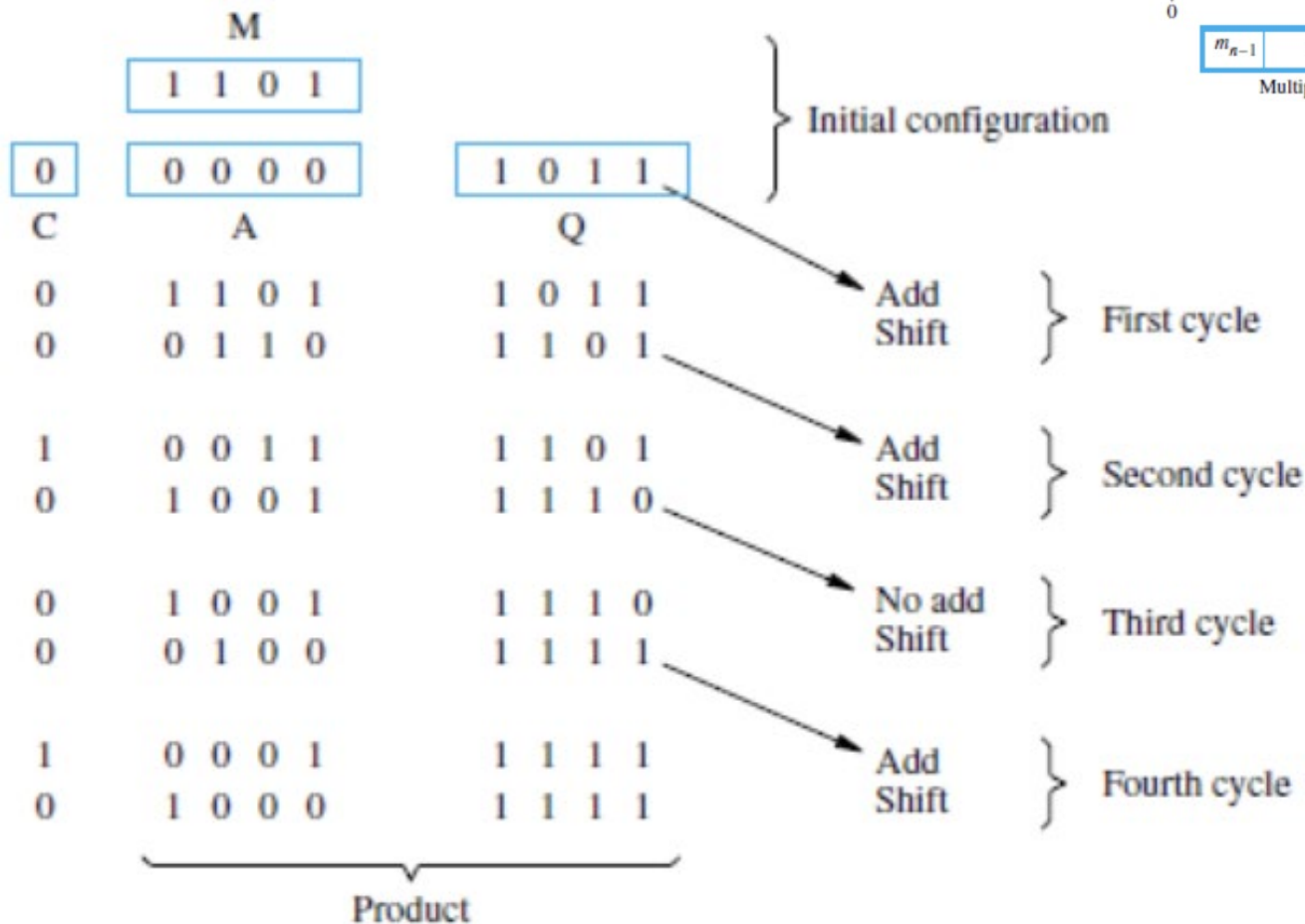


Sequential Circuit for Binary Multiplier



$$\begin{array}{r}
 1101 \\
 \times 1011 \\
 \hline
 1101 \\
 0000 \\
 0000 \\
 0000 \\
 \hline
 10001111
 \end{array}$$

Multiplier Example



Signed Multiplication (1)

Considering 2's-complement signed operands, what will happen to $(-13) \times (+11)$ if following the same method of unsigned multiplication?

Note that negative multiplicand

					1	0	0	1	1	(- 13)
					0	1	0	1	1	(+ 11)
	1	1	1	1	1	0	0	1	1	
	1	1	1	1	1	0	0	1	1	
	0	0	0	0	0	0	0	0		
	1	1	1	0	0	1	1			
	0	0	0	0	0					
	1	1	0	1	1	1	0	0	0	1 (- 143)

Sign extension is shown in red



Signed Multiplication (2)

- For a negative multiplier, a straightforward solution is to form the 2's-complement of both the multiplier and the multiplicand and proceed as in the case of a positive multiplier.
- This is possible because complementation of both operands does not change the value or the sign of the product.

Booth Algorithm (1)

- Scan the multiplier from right to left
 - If the digit changes from 0 to 1, then multiply '-1' to the shifted multiplicand.
 - If the digit changes from 1 to 0, then multiply '+1' to the shifted multiplicand.

Multiplier 0 0 1 1 1 1 0

New multiplier 0 +1 0 0 0 -1 0

Note, instead of multiplying with 30 (0011110), we can multiply with 32-2 (0100000 – 0000010)

Booth Algorithm (2)

- The booth algorithm can benefit most when the multiplier consist of a block of 1
- The worst case scenario is when the multiplier has alternate pattern of 0 and 1.

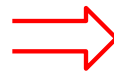
0 1 0 1 0 1 0 1 0 1 0 1

								0	1	0	1	1	0	1
								0	+1	0	0	0	-1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	0	1	0	0	1	1	←
0	0	0	0	0	0	0	0	0	0	0	0	0		
0	0	0	0	0	0	0	0	0	0	0	0			
0	0	0	0	0	0	0	0	0	0	0				
0	0	0	1	0	1	1	0	1						
0	0	0	0	0	0	0	0							
0	0	0	1	0	1	0	1	0	0	0	1	1	0	

								0	1	0	1	1	0	1
								0	0	+1	+1	+1	+1	0
								0	0	0	0	0	0	0
							0	1	0	1	1	0	1	
						0	1	0	1	1	0	1		
					0	1	0	1	1	0	1			
				0	1	0	1	1	0	1				
			0	0	0	0	0	0	0					
		0	0	0	0	0	0							
0	0	0	1	0	1	0	1	0	0	0	1	1	0	

Booth Algorithm with Negative Multiplier

$$\begin{array}{r}
 0 \ 1 \ 1 \ 0 \ 1 \quad (+13) \\
 \times 1 \ 1 \ 0 \ 1 \ 0 \quad (-6) \\
 \hline
 \end{array}$$



$$\begin{array}{r}
 0 \ 1 \ 1 \ 0 \ 1 \\
 0 \ -1 \ +1 \ -1 \ 0 \\
 \hline
 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\
 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \\
 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \\
 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \\
 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\
 \hline
 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \quad (-78)
 \end{array}$$

Fast Multiplication

- Bit-pair recoding multiplication
 - Bit-pair recoding halves the maximum number of sum
- Carry-Save Addition
 - Achieved by saving the carry bit and feeding it directly to adders in the next row at correct positions.
- Addition Tree using Reducer
 - Group the summands in threes and perform CSA on each groups in parallel.

Bit-pair Recoding

Idea is ..

pair (+1, -1) $\equiv$ pair (0, +1)

pair (-1, +1) $\equiv$ pair (0, -1)

pair (+1, 0) $\equiv$ pair (0, +2)

$$\begin{array}{r} 0 \ 1 \ 1 \ 0 \ 1 \ (+13) \\ 1 \ 1 \ 0 \ 1 \ 0 \ (-6) \\ \hline \end{array}$$



$$\begin{array}{r} 0 \ 1 \ 1 \ 0 \ 1 \\ 0 \ -1 \ +1 \ -1 \ 0 \\ \hline 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \\ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\ \hline 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ (-78) \end{array}$$

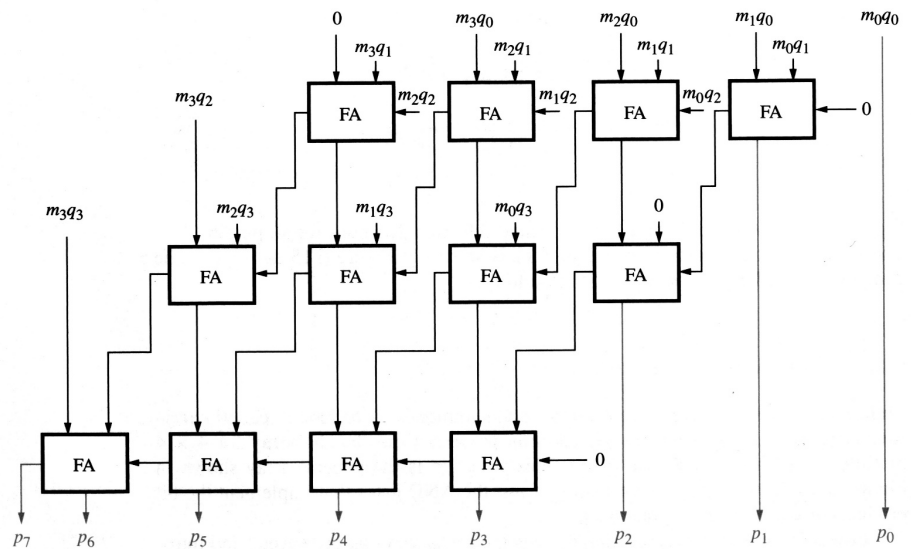
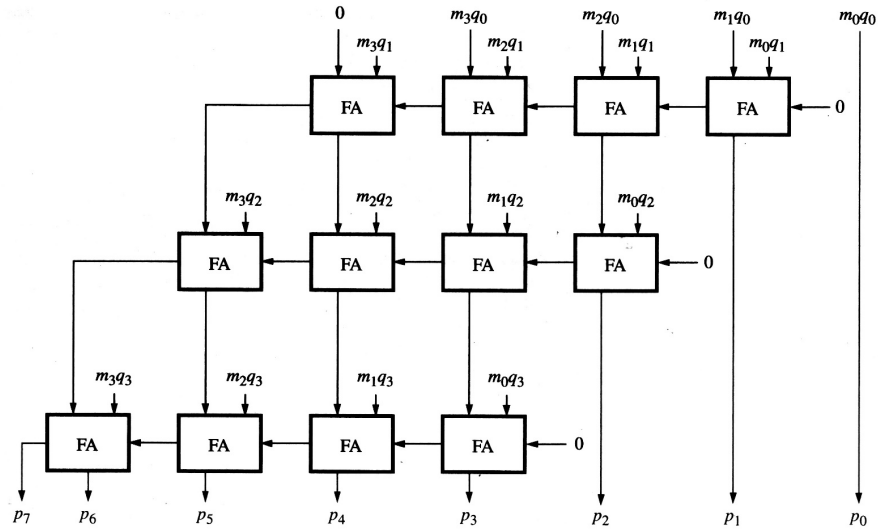


$$\begin{array}{r} 0 \ 1 \ 1 \ 0 \ 1 \\ 0 \ 1 \ 2 \\ \hline 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \\ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\ \hline 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \end{array}$$

Table of Multiplicand Selection

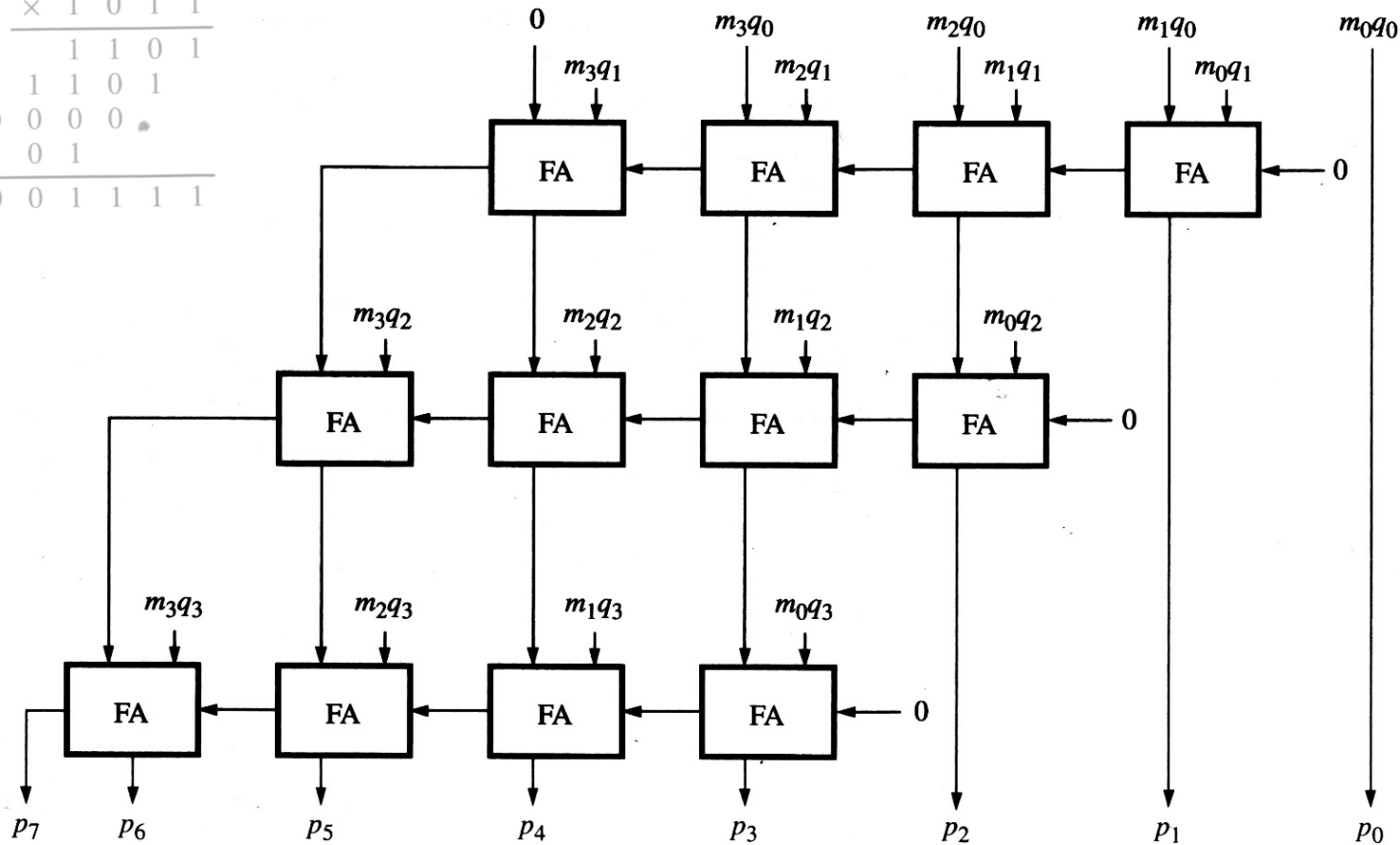
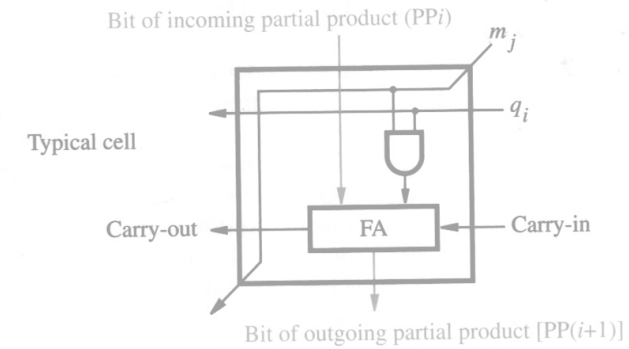
Multiplier bit-pair		Multiplier bit on the right $i - 1$	Multiplicand selected at position i	
$i + 1$	i			
0	0	0	0	X M
0	0	1	+ 1	X M
0	1	0	+ 1	X M
0	1	1	+ 2	X M
1	0	0	- 2	X M
1	0	1	- 1	X M
1	1	0	- 1	X M
1	1	1	0	X M

Carry-Save Addition (CSA)



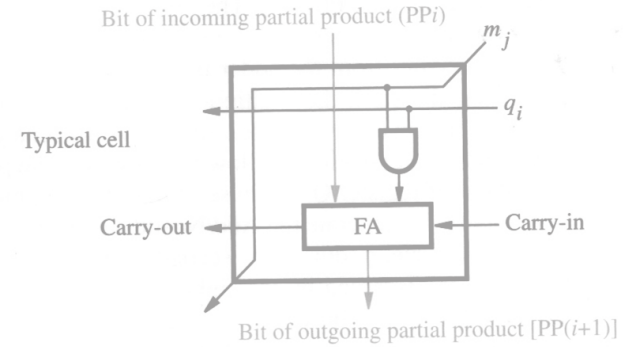
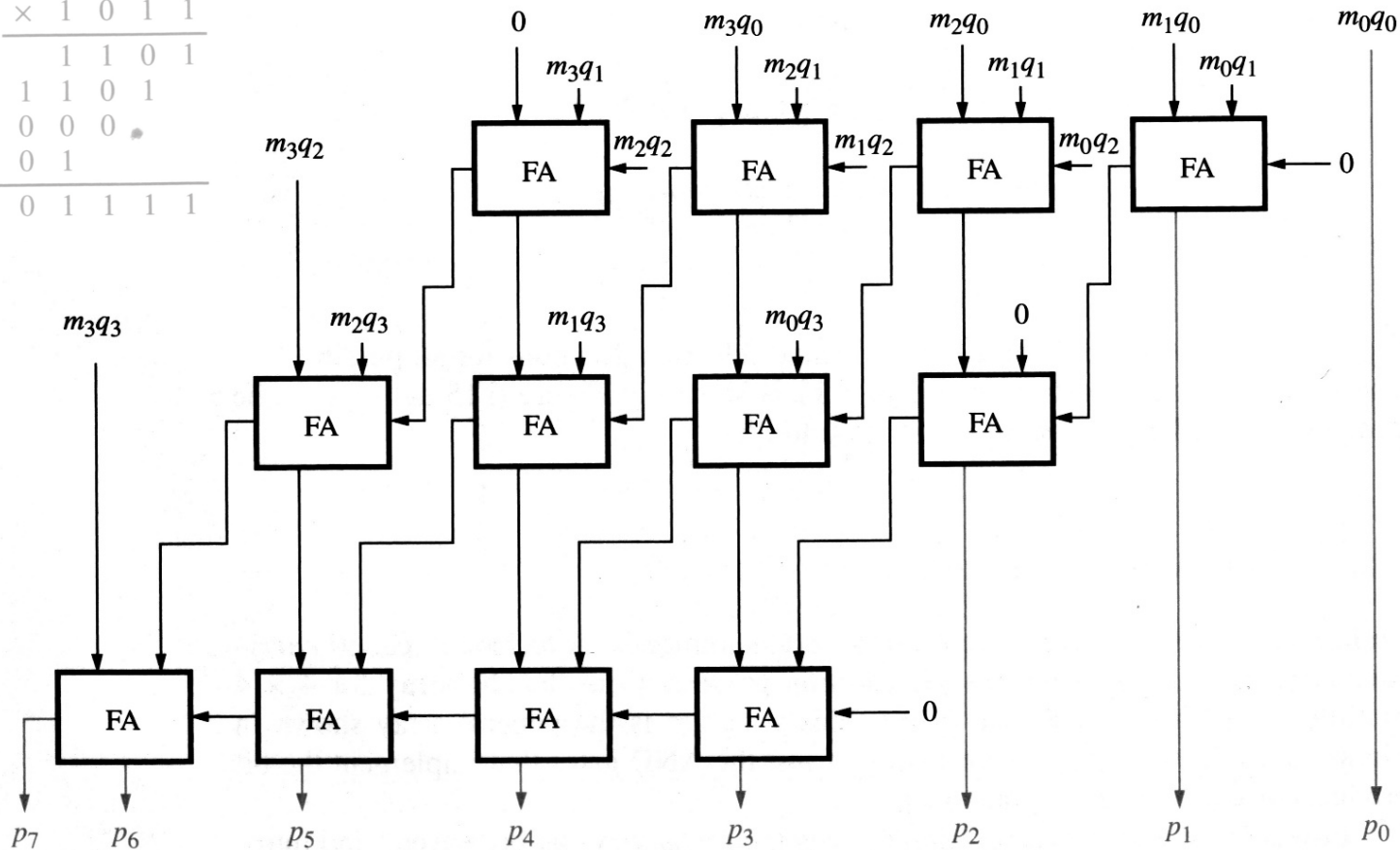
Circuit of Carry Ripple Array

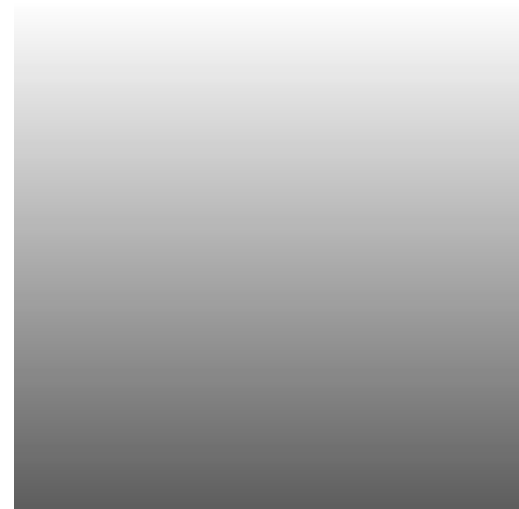
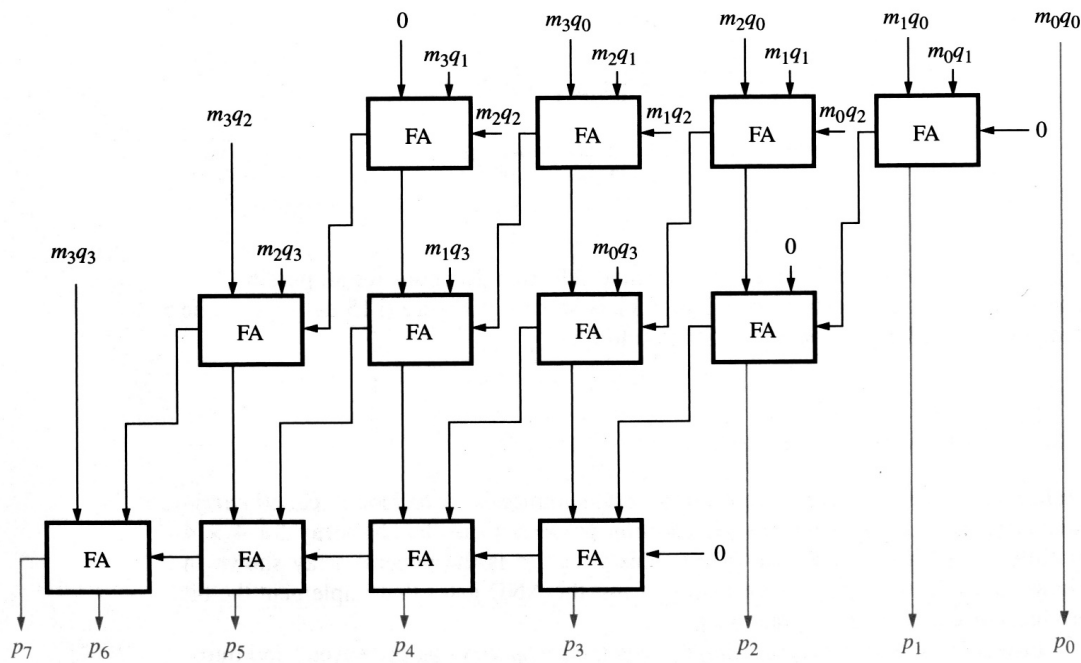
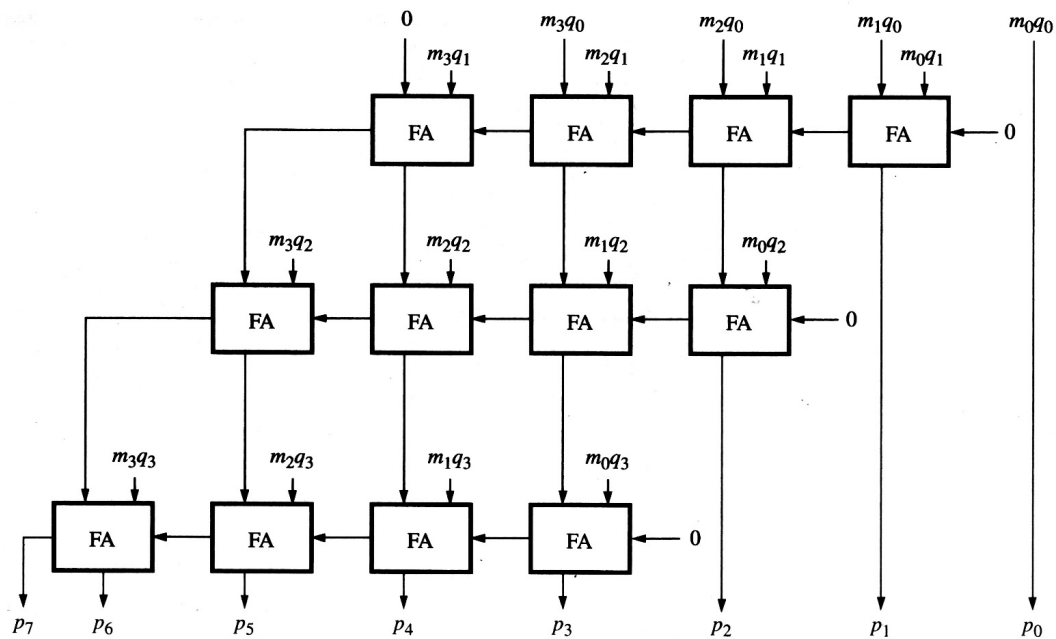
$$\begin{array}{r}
 1101 \\
 \times 1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 10001111
 \end{array}$$



Circuit of Carry Save Addition

$$\begin{array}{r}
 1101 \\
 \times 1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 10001111
 \end{array}$$





Addition Tree with Reducers

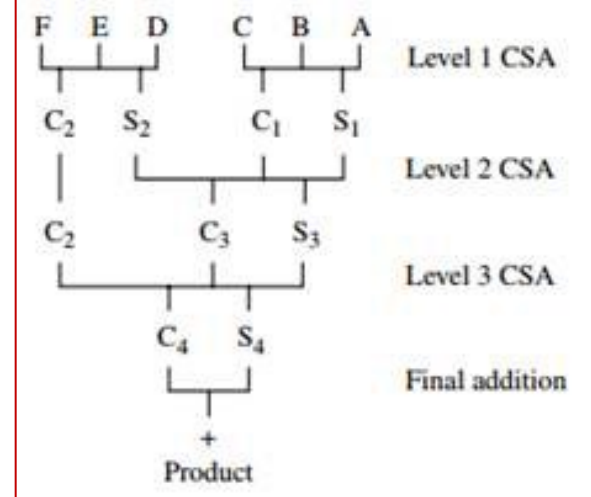
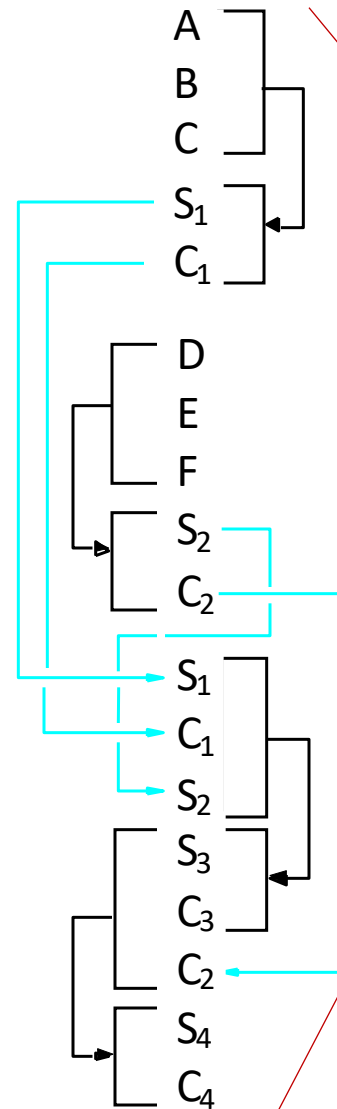
- Group the summands in threes and perform carry-save addition on each of these groups in parallel to generate a set of S and C vectors in one full-adder delay
- Group all of the S and C vectors into threes, and perform carry-save addition on them, generating a further set of S and C vectors in one more full-adder delay
- Continue with this process until there are only two vectors remaining
- They can be added in a RCA or CSA to produce the desired product

Carry Save Addition (CSA) using 3-2 Reducers

						1	0	1	1	0	1	(45)	M
					x	1	1	1	1	1	1	(63)	Q
						1	0	1	1	0	1	A	
				1		0	1	1	0	1		B	
			1	0		1	1	0	1			C	
				1	0	1	1	0	1			D	
			1	0	1	1	0	1				E	
		1	0	1	1	0	1					F	
1	0	1	1	0	0	0	1	0	0	1	1	(2,835)	Product

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 1\ M \\
 \times 1\ 1\ 1\ 1\ 1\ 1\ Q \\
 \hline
 \end{array}$$

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 1 \\
 1\ 0\ 1\ 1\ 0\ 1 \\
 1\ 0\ 1\ 1\ 0\ 1 \\
 \hline
 1\ 1\ 0\ 0\ 0\ 0\ 1\ 1 \\
 0\ 0\ 1\ 1\ 1\ 0\ 0 \\
 \hline
 1\ 0\ 1\ 1\ 0\ 1 \\
 1\ 0\ 1\ 1\ 0\ 1 \\
 1\ 0\ 1\ 1\ 0\ 1 \\
 \hline
 1\ 1\ 0\ 0\ 0\ 0\ 1\ 1 \\
 0\ 0\ 1\ 1\ 1\ 0\ 0 \\
 \hline
 1\ 1\ 0\ 0\ 0\ 0\ 1\ 1 \\
 1\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \\
 0\ 0\ 0\ 0\ 1\ 0\ 1\ 1\ 0\ 0\ 0 \\
 0\ 0\ 1\ 1\ 1\ 1\ 0\ 0 \\
 \hline
 0\ 1\ 0\ 1\ 1\ 1\ 0\ 1\ 0\ 0\ 1\ 1 \\
 + 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0\ 0 \\
 \hline
 1\ 0\ 1\ 1\ 0\ 0\ 0\ 1\ 0\ 0\ 1\ 1\ \text{Product}
 \end{array}$$



Manual Division: Longhand Division

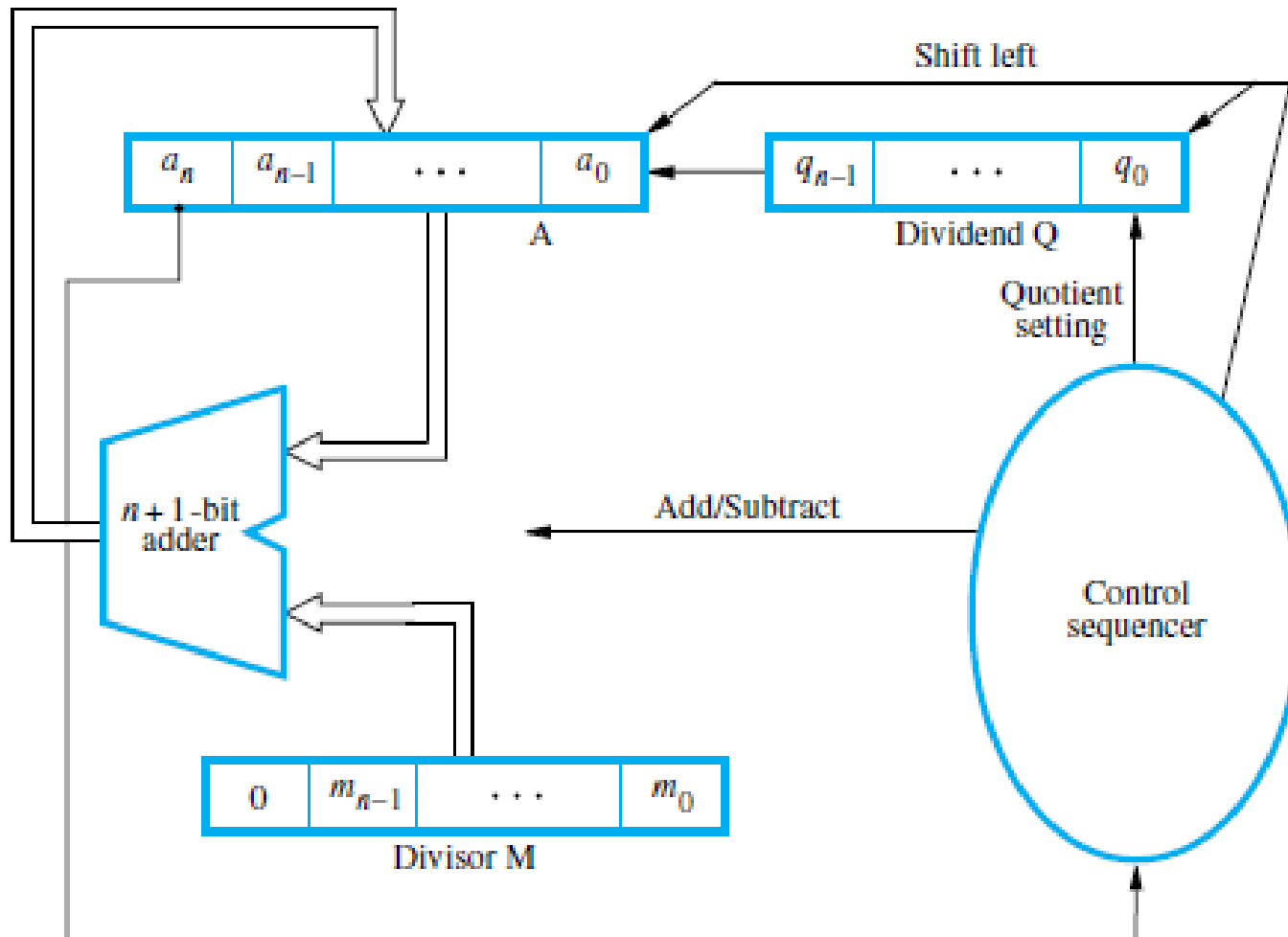
$$\begin{array}{r} 21 \\ 13 \overline{) 274} \\ \underline{26} \\ 14 \\ \underline{13} \\ 1 \end{array}$$

$$\begin{array}{r} 10101 \\ 1101 \overline{) 100010010} \\ \underline{1101} \\ 10000 \\ \underline{1101} \\ 1110 \\ \underline{1101} \\ 1 \end{array}$$

Longhand Division Steps

- Position the divisor appropriately with respect to the dividend and performs a subtraction.
- If the remainder is zero or positive, a quotient bit of 1 is determined, the remainder is extended by another bit of the dividend, the divisor is repositioned, and another subtraction is performed.
- If the remainder is negative, a quotient bit of 0 is determined, the dividend is restored by adding back the divisor, and the divisor is repositioned for another subtraction.

Sequential Circuit for Binary Division



Restoring Division (1)

$$\begin{array}{r}
 11 \overline{) 1000} \\
 \underline{11} \\
 10
 \end{array}$$

Initially

	A	Q
0 0 0 0 0		1 0 0 0
	M	
0 0 0 1 1		

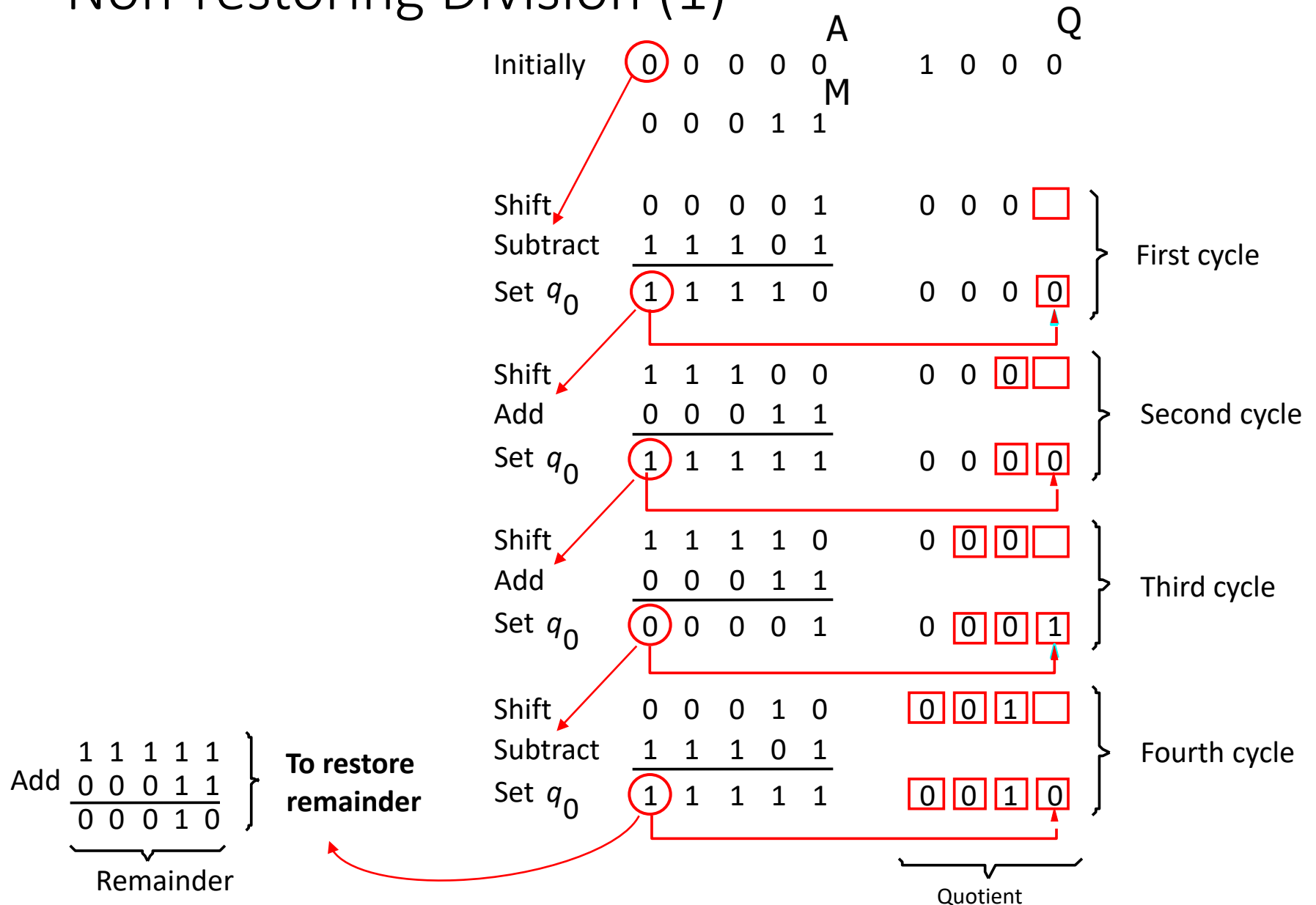
Shift	0 0 0 0 1	0 0 0 	} First cycle
Subtract	<u>1 1 1 0 1</u>		
Set q_0	1 1 1 1 0		
Restore	<u>1 1</u>		
	0 0 0 0 1	0 0 0 0	} Second cycle
Shift	0 0 0 1 0	0 0 0 	
Subtract	<u>1 1 1 0 1</u>		
Set q_0	1 1 1 1 1		
Restore	<u>1 1</u>		} Third cycle
	0 0 0 1 0	0 0 0 0	
Shift	0 0 1 0 0	0 0 0 	
Subtract	<u>1 1 1 0 1</u>		
Set q_0	0 0 0 0 1		} Fourth cycle
Restore	None		
	0 0 0 0 1	0 0 0 1	
Shift	0 0 0 1 0	0 0 1 	
Subtract	<u>1 1 1 0 1</u>		} Fifth cycle
Set q_0	1 1 1 1 1		
Restore	<u>1 1</u>		
	0 0 0 1 0	0 0 1 0	

Remainder
Quotient

Restoring Division (2)

- 1) Shift A and Q left one binary position
- 2) Subtract M from A, and place the answer back in A
- 3) If the sign of A is 1, set q_0 to 0 and add M back to A (restore A); otherwise, set q_0 to 1
- 4) Repeat these steps n times

Non-restoring Division (1)

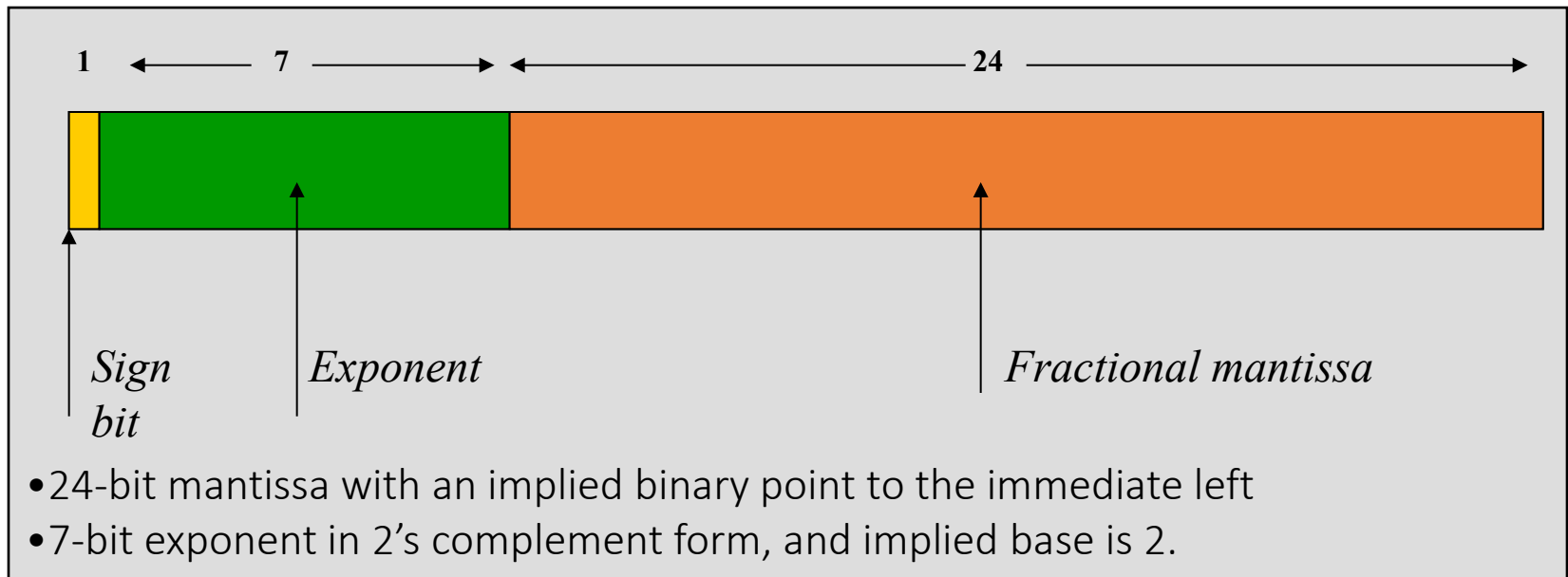


Non-restoring Division (2)

- Avoid the need for restoring A after an unsuccessful subtraction.
- Part 1: Repeat n times
 - 1) If the sign of A is 0, shift A and Q left one bit position and subtract M from A;
 - 2) otherwise, shift A and Q left and add M to A.
 - 3) Now, if the sign of A is 0, set q_0 to 1; otherwise, set q_0 to 0.
- Part 2: If the sign of A is 1, add M to A

We learn about integer number and operation!
What 's about floating point number and operation?

A sample representation



Supplementary Section

Example of Bit-pair Recoding

$$\begin{array}{r} -4 \\ \times 5 \\ \hline -20 \end{array}$$

[illegible]